

Predicting US Flight Delays: A Two-Stage Machine Learning Approach with Explicit Data Leakage Analysis

Nayan Nitin Kadhre
College of Information Science
University of Arizona
Tuscon, AZ, US
nayank@arizona.edu

Young Soo Ko
College of Information Science
University of Arizona
Tuscon, AZ, US
yjrsooko@arizona.edu

Jenna Rose Childress
Department of Systems & Industrial
Engineering
University of Arizona
Tuscon, AZ, US
jrchild4@arizona.edu

Abstract— Flight delays affect roughly one in five US domestic flights, imposing substantial costs on passengers and airlines. This project develops a two-stage machine learning system using the "US 2023 Civil Flights" Kaggle dataset (~6.7M flights joined with airport-level weather), framing the problem as binary classification of delays exceeding the FAA-standard fifteen-minute threshold followed by regression of delay duration. To produce a realistic assessment of pre-flight predictability, every model is evaluated under two configurations — one retaining all features and one removing seven post-arrival columns that constitute target leakage. Four classifiers and four regressors were implemented in PySpark on Databricks. With leakage included, all classifiers achieve AUC-ROC near 0.999, but after removal, Random Forest leads with AUC-ROC of 0.665 and single-stage regression collapses to $R^2 \approx 0.03$. To address this, a two-stage hybrid architecture separates severe-delay outliers (>143 minutes) from the normal range, achieving an MAE of 20.75 minutes — a substantial improvement over the ~49-minute baseline. The study's central contribution is the explicit quantification of the gap between leakage-inflated and realistic pre-flight performance.

Keywords— *Flight delay prediction, Data leakage, Random Forest, Two-stage prediction, PySpark, Feature engineering*

I. INTRODUCTION

Flight delays remain one of the most persistent and costly disruptions in commercial aviation. Across the United States, roughly one in five domestic flights departs more than fifteen minutes behind schedule [1], generating cascading impacts on passengers, airlines, and downstream airport operations [2]. For travelers, even a moderate delay can mean missed connections, lost ground transportation, or rebooked accommodations; for carriers, delays translate into crew rescheduling, gate congestion, and significant operational cost [3]. Despite the maturity of modern aviation systems, accurately anticipating these disruptions before departure remains a difficult prediction problem because delays are driven by a complex interaction of weather, air-traffic dynamics, carrier-specific factors, and aircraft routing histories [4].

This paper addresses the question of whether departure delays can be reliably predicted using only information available at or before scheduled departure time. We frame the problem as a two-stage learning task. First, a binary classification model determines whether a given flight will be delayed beyond the FAA-standard fifteen-minute threshold. Second, conditional on a flight being predicted as delayed, a

regression model estimates the expected delay duration in minutes [3]. Beyond predictive modeling, the project also surfaces actionable trends — by airline, time of day, day of week, season, and airport — that travelers can use to make better booking decisions.

A central methodological theme of this work is the deliberate identification and removal of data leakage. Many publicly available flight datasets contain post-arrival fields such as Arr_Delay and per-cause delay breakdowns that are observable only after a flight has landed. Including these features produces near-perfect predictive scores that are entirely unrealistic for a real-time deployment, compromising the reliability of the models [5]. We therefore evaluate every model under two configurations — one with all features included and one with all leakage features removed — to quantify the true pre-flight predictive signal and to give an honest assessment of what is and is not learnable from the available data.

II. RELATED WORK

Flight delay prediction has been studied extensively in both academic and industry settings, with approaches ranging from statistical time-series models to deep learning architectures [3]. Earlier work emphasized airline-level operational factors and weather covariates, often using logistic regression or shallow tree ensembles to estimate delay probability at the route or carrier level [6]. More recent literature has expanded into gradient boosting, neural networks, and graph-based models that exploit the network structure of airport connections [7], [8]. These modern architectures increasingly incorporate real-time air traffic flow data and aircraft turnaround histories to capture complex spatial and temporal dependencies across the aviation network [3], [8].

A recurring theme in the literature is the gap between reported model accuracy and real-world deployability. Many published studies achieve high reported scores by inadvertently using post-flight features as predictors, a form of target leakage that inflates apparent performance [5]. Recent work has emphasized the importance of strict feature-time alignment—ensuring that models only use information that would have been available at the exact moment of prediction to maintain benchmarking reliability [5]. Our project follows this tradition by explicitly separating leakage and no-leakage tracks, evaluating realistic pre-flight

prediction performance rather than optimizing for inflated, unrealistic scores.

III. DATASET AND DATA SOURCES

The data used in this project comes from the publicly available Kaggle dataset "US 2023 Civil Flights, delays, meteo and aircrafts," [9] which compiles US domestic flight, airport, weather, and aircraft information for the 2023 calendar year. Because the dataset is fully public and contains no personally identifiable information about passengers, the project does not require IRB review.

The dataset merges 2023 US domestic flight records (US_flights_2023.csv) with daily weather (weather_meteo_by_airport.csv) for the departure airport. Cancelled, diverted, and redundant 2024 data were excluded. This integrated dataset captures the complete pre-departure context (schedule, route, aircraft, and weather) alongside post-arrival outcomes (delay durations and causes) required to train and evaluate real-time prediction models.

A. Dataset with Leakage Features

The "with leakage" configuration includes seven post-arrival columns (e.g., Arr_Delay and specific delay causes). Because these are recorded only after landing, they are unavailable for real-time prediction and serve solely as a baseline to quantify the impact of data leakage. The regression set does not include Arr_Delay since this is its target variable.

B. Dataset without Leakage Features

Conversely, the "without leakage" configuration strictly removes these post-arrival features. It relies exclusively on data observable before departure, such as schedules, aircraft details, and local weather. This reflects a realistic real-time deployment scenario and forms the primary basis for our model evaluation.

IV. TRENDS AND FLIGHT RECOMMENDATIONS

Before turning to predictive modeling, we conducted an exploratory analysis of the joined dataset to surface high-level patterns useful both for feature engineering and for direct passenger guidance. Across approximately 6.7 million flights in 2023, roughly one in five flights was delayed by more than fifteen minutes, with a mean arrival delay of 6.6 minutes. Although the average is modest, the distribution is heavy-tailed, with a small fraction of flights driving a disproportionate share of total delay minutes.

Several temporal patterns emerged clearly. Afternoon and evening departures are substantially more delay-prone than morning flights, consistent with the well-documented compounding effect of upstream delays as the day progresses. By day of week, Fridays and Sundays show the highest delay rates, while mid-week flights perform comparatively better (Figure 1). Seasonally, June and July stand out as the highest-risk months, reflecting both summer thunderstorm activity and peak travel volume.

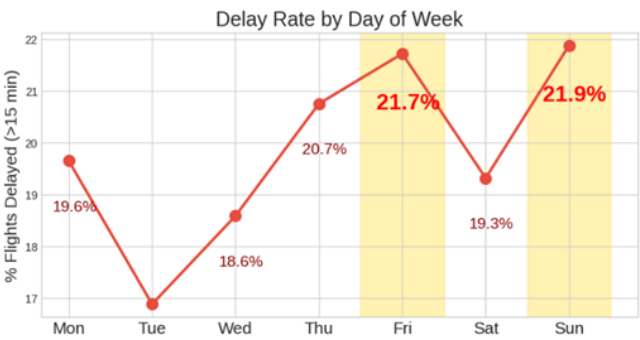


Figure 1. Delay rate by day of week.

Weather drivers are equally pronounced. As shown in Figure 2, heavy precipitation roughly doubles the delay probability, from a baseline near 17% to approximately 38% under extreme rainfall conditions. Storm-force winds are similarly strong delay triggers. By contrast, snow events — though intuitively associated with disruption — appear less frequently in the 2023 data and contribute proportionally less to overall delay minutes than precipitation.

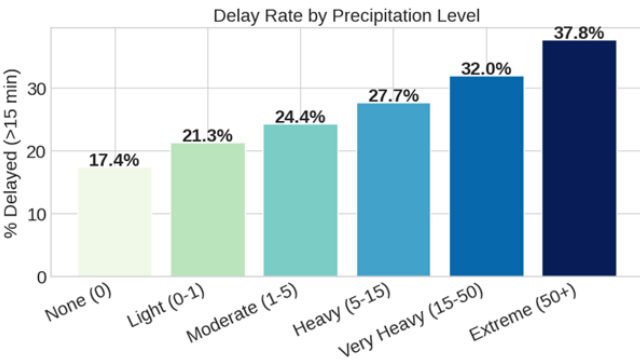


Figure 2. Delay rate by precipitation level.

As shown in the airline-specific analysis results in Figure 3, consistent differences in airline performance were revealed. JetBlue and Frontier emerge as persistent under-performers, with delay rates well above the cross-carrier average, while Republic Airways and Endeavor Air consistently arrive ahead of schedule on average.

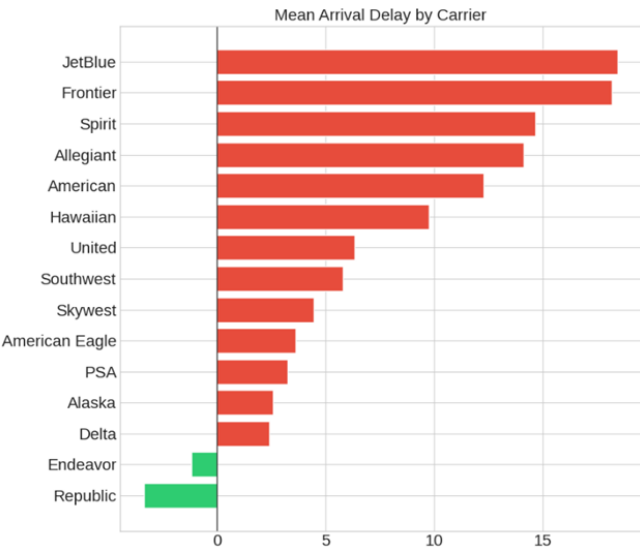


Figure 3. Mean arrival delay by carrier.

As illustrated in Figure 4, an airport-by-airport analysis reveals that major airports in the Florida region—specifically Fort Lauderdale (FLL, 30.4%), Orlando (MCO, 27.9%), and Miami (MIA, 26.5%)—occupy the top spots in delay rankings, collectively forming the group with the poorest performance. Conversely, medium-sized airports in the Midwest and Mountain West regions (GSP 13.2%, GSO 13.8%, TUL 14.0%) are leading the pack in on-time performance, recording delay rates that are roughly half those of the major Florida airports. By delay cause, Last-Aircraft and Carrier delays jointly account for approximately 75% of total delay time, while Weather delays have the highest per-incident severity but a lower base rate.

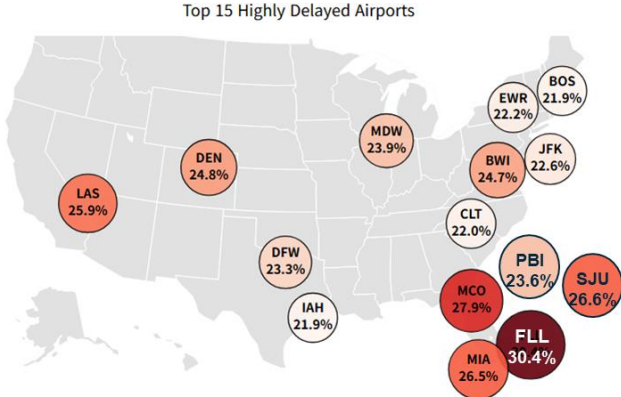


Figure 4. Top 15 Highly delayed airports.

Translated into actionable guidance: passengers seeking to minimize delay risk should prefer morning departures on mid-week days, avoid the June–July peak, choose carriers with strong on-time records (Republic, Endeavor) over chronic under-performers, and — where the route allows — favor mid-sized Midwest or Mountain West connection airports over Florida hubs.

V. DATA PREPROCESSING AND FEATURE ENGINEERING

The preprocessing pipeline was implemented in PySpark [10] on Databricks [11] and is organized into two stages: a cleaning stage that produces a consistent and complete record-level table, and a feature engineering stage that derives the modeling features and the binary classification target.

A. Data Cleaning

The cleaning pipeline began by removing redundant columns introduced during the dataset join, specifically the time and airport id columns, which were no longer needed after merging. Flights with a recorded duration of zero or null were then removed to eliminate invalid records, resulting in the removal of only one such row.

Rows with missing values in the Dep_Delay and Arr_Delay columns were dropped, as these are essential for both the classification and regression tasks. To handle extreme outliers in the delay columns, a capping strategy was applied: delays greater than 1,440 minutes (24 hours) or less than -180 minutes were capped at those thresholds to remove unrealistic values while preserving the overall distribution.

For missing values across the remaining delay columns, null or NA entries were filled with zero, operating under the assumption that no recorded delay indicates no delay

occurred. Similarly, weather-related columns such as snow and precipitation were filled with zero for missing values. All other numeric columns with missing values were imputed using the column mean. The aircraft age column was treated separately, where both null values and negative values were replaced with the column mean, as a negative aircraft age is not physically meaningful.

Following imputation, all columns were renamed to be consistent and descriptive. Finally, all duplicate rows and any remaining redundant columns were dropped to produce a clean dataset ready for feature engineering.

B. Feature Engineering

The first step in feature engineering was creating the binary classification label. A new column called "Delayed (> 15 min)" was derived from the departure delay column, assigned a value of 1 if the departure delay exceeded 15 minutes and 0 otherwise, in alignment with the FAA standard [12] definition of a flight delay.

Temporal features were then extracted from the flight date column. The day of the month and month were created as separate numeric columns to allow the model to capture seasonal and intra-month patterns in delays. Additionally, the departure time label was encoded into a numeric representation using the following mapping: Morning = 0, Afternoon = 1, Evening = 2, and Night = 3. After extraction, the original flight date and departure time label columns were dropped as they were no longer needed.

A correlation heatmap was generated to identify multicollinearity among features. Based on this analysis, the columns tmin and tmax were removed due to their high correlation with one another, which makes it difficult to isolate the individual effect of each feature on the target variable. Tavg was still included to prevent losing necessary temperature information. Additionally, the columns Tail_Number and Manufacturer were removed as they were deemed non-informative for delay prediction and manufacturer information was already included in the Model column.

Finally, all remaining categorical features were converted to numeric representations using one-hot encoding. The original categorical columns and their intermediate indexed columns were dropped after encoding, leaving a fully numeric feature set suitable for machine learning model training.

VI. METHODOLOGY

We approach flight delay prediction as a two-stage supervised learning problem. The first stage is a binary classification task that predicts whether a given flight will be delayed by more than fifteen minutes. The second stage is a regression task that, conditional on a flight being delayed, predicts the magnitude of the delay in minutes. Both stages are evaluated under the with-leakage and without-leakage feature configurations described in Section III, allowing us to isolate the contribution of legitimate pre-flight features.

A. Classification Model

The classification task addresses the binary question: will a given flight be delayed? The target variable, Delayed (> 15 min), takes a value of 1 for delayed flights and 0 for on-time flights. Four supervised learning algorithms were selected to

cover a range of model complexity and interpretability, allowing for a meaningful comparison across both predictive performance and practical usability. The four classifiers used in this study are described below:

- Decision Tree - splits on the most discriminative feature thresholds at each node, chosen for transparency and interpretability of which pre-flight factors drive delays.
- Random Forest - ensemble of 30 trees with bootstrapped sampling and random feature subsets, reducing variance while retaining feature importance rankings.
- Logistic Regression - models log-odds of delay as a regularized (L_2 , $\lambda=0.01$) linear combination of StandardScaler-normalized features, enabling direct coefficient interpretation.
- Linear SVM - finds the maximum-margin hyperplane in the scaled feature space, providing strong generalization and a non-probabilistic complement to the other three models.

All four models were trained and evaluated under two distinct feature configurations to isolate the impact of data leakage. The All Features track includes the full cleaned schema, which retains post-arrival columns such as Arr_Delay, Delay_Carrier, Delay_Weather, and related breakdown columns. These fields are only observable after a flight has landed and therefore constitute direct leakage of the outcome. The No Leakage Features track removes these seven columns before assembly, leaving only information available at or before departure time. This split enables a direct quantification of how much predictive signal is attributable to legitimate pre-flight factors versus post-hoc leakage.

B. Regression Model

The regression models look at delayed flights and predict exactly how long they will be delayed. To capture the full problem, this step takes the output of the classification task to only predict the delay length for flights that are predicted to be delayed. Four models were selected that are similar to those from the regression task:

- Decision Tree - splits based on node impurity, creating understandable decision rules for prediction while handling nonlinear complexities in the data.
- Random Forest - uses bagging to average the results of 30 decision trees, resulting in more stable predictions.
- Gradient Boosted Trees (GBT) - uses boosting to improve model performance through 50 iterations of decision trees training to correct previous prediction errors.
- Linear Regression - simplest regression model that models linear relationships on the standardized features with L_2 regularization and has high interpretability.

Similar to the classification task, these four models were trained using two different sets of features. One has all of the available features except for Delayed (> 15 min). The other has information about specific delays that occur throughout the flight removed. In both cases, the target variable is Arr_Delay. There is less of a leakage problem than the classification task since Delayed (> 15 min) and Arr_Delay are not included as predictors. Some of the delay-specific

variables may not be provided until after the flight arrives, but it may be possible to track some of them if this version of the features has high performance.

In addition to the four base regressors, we also implemented an iterative improvement track that combined cyclical encoding of temporal features, target encoding of high-cardinality categoricals, and Bayesian hyperparameter optimization (Optuna [13], 30 trials) on a LightGBM [14] regressor. Finally, motivated by the heavy tail of the delay distribution, we developed a two-stage hybrid system in which a classifier first separates "severe delay" outliers (>143 minutes, the top 10% of delayed flights) from the normal range, and a dedicated regressor predicts the exact duration only on the normal-range subset.

VII. ENVIRONMENTAL SETUP

A. Tools and Environment

All data processing and modeling were carried out in PySpark on the free tier of Databricks Community Edition. PySpark was chosen over single-node Python tooling because the joined dataset exceeds the comfortable working size of a typical pandas workflow, and because PySpark MLlib provides distributed implementations of all four classifiers and all four regressors used in the study under a single consistent API. Visualization and exploratory analysis were performed in matplotlib and seaborn within the Databricks notebook environment, and the LightGBM iterative-improvement track was implemented using the standard Python lightgbm library combined with Optuna for Bayesian hyperparameter search.

B. Dataset Split

To allow for accurate comparisons across the classification and regression tasks and each of their models, the same train-test split was used in each instance. The split was an 80/20 split so there would be sufficient data for both training and testing. For the regression task, the train and test sets were filtered to include only delayed flights, matching the defined task of only predicting delay duration for delayed flights. In addition, the data was standardized when used for logistic regression, linear SVM, and linear regression to prevent features with larger values from dominating the results.

C. Evaluation Metrics

For classification models, model performance was assessed using PySpark's Binary Classification Evaluator and Multiclass Classification Evaluator across both train and test sets. Four metrics were computed: Accuracy (proportion correct, reported for completeness), Weighted F1 (harmonic mean of precision and recall, preferred under class imbalance), Weighted Precision and Recall (per-class averages weighted by support), and AUC-ROC - the primary comparison metric, as it measures discriminative ability across all classification thresholds and is robust to class imbalance.

The regression models used PySpark's Regression Evaluator to compare performance across the train and test sets using three metrics. The first was root mean square error (RMSE), which is sensitive to outliers and signifies if there are any extreme predictions. Second is mean average error (MAE), which shows the average prediction error in minutes.

Finally, R^2 corresponds to the percent of variation in delay explained by the model.

D. Hyperparameter Tuning

Hyperparameters for the four base classifiers and four base regressors were selected through a constrained manual sweep informed by initial runs. Tree-based models were configured with moderate depth limits (typically $\text{maxDepth} = 10$) and modest ensemble sizes ($\text{numTrees} = 30$ for Random Forest) to balance fit quality against the computational ceiling of the free Databricks tier. Logistic regression used L_2 regularization with $\lambda = 0.01$, applied to features standardized by StandardScaler. Linear SVM and linear regression used similar parameters with $\lambda = 0.1$. A full grid search across all parameter combinations was not feasible given the dataset size and free-tier compute limits; this is acknowledged as a limitation in Section VIII. The LightGBM iterative-improvement track used Optuna with 30 trials of Bayesian optimization over learning rate, number of leaves, max depth, minimum data in leaf, feature fraction, and bagging fraction.

VIII. RESULT AND EVALUATION

A. Classification Results (With Leakage Features)

When all features are retained, every model achieves near-perfect performance (Tables 1 and 2). These results reflect the presence of severe data leakage rather than genuine predictive ability.

Table 1. Training set performance for classification, with leakage features.

Model	AUC-ROC	Accuracy	F1	Precision	Recall
Decision Tree	0.999	0.997	0.997	0.997	0.997
Random Forest	0.999	0.981	0.981	0.982	0.981
Logistic Regression	0.999	0.963	0.962	0.964	0.963
Linear SVM	0.999	0.934	0.929	0.939	0.934

Table 2. Test set performance for classification with all features (leakage included).

Model	AUC-ROC	Accuracy	F1	Precision	Recall
Decision Tree	0.999	0.997	0.997	0.997	0.997
Random Forest	0.999	0.982	0.981	0.982	0.982
Logistic Regression	0.999	0.963	0.962	0.965	0.963
Linear SVM	0.999	0.934	0.929	0.939	0.934

All four models achieve near-perfect scores in this configuration, driven entirely by leakage rather than genuine predictive ability. Near-identical train and test scores rule out overfitting; the models are simply exploiting post-arrival columns like `Arr_Delay` that directly encode the outcome.

B. Classification Results (Without Leakage Features)

After removing the seven leakage columns - `Arr_Delay`, `Dep_Delay`, `Delay_Carrier`, `Delay_Weather`, `Delay_NAS`, `Delay_Security`, and `Delay_LastAircraft` - model performance drops substantially, reflecting the true difficulty of predicting departure delay from pre-flight information. Tables 3 and 4 show the train and test performance, respectively.

Table 3. Training set performance for classification, without leakage features.

Model	AUC-ROC	Accuracy	F1	Precision	Recall
Decision Tree	0.532	0.804	0.733	0.766	0.804
Random Forest	0.665	0.801	0.712	0.641	0.801
Logistic Regression	0.665	0.801	0.719	0.743	0.801
Linear SVM	0.619	0.801	0.712	0.774	0.801

Table 4. Test set performance for classification without leakage features (realistic pre-flight scenario).

Model	AUC-ROC	Accuracy	F1	Precision	Recall
Decision Tree	0.533	0.805	0.733	0.764	0.805
Random Forest	0.665	0.802	0.714	0.643	0.802
Logistic Regression	0.665	0.802	0.720	0.742	0.802
Linear SVM	0.620	0.802	0.714	0.643	0.802

Accuracy clusters around 80% for all models, consistent with a class-imbalanced dataset where predicting on-time for every flight would yield similar results, making AUC-ROC the more meaningful metric here. Random forest leads with an AUC of 0.6655, closely followed by logistic regression (0.6659, a marginal difference), but random forest's ability to capture non-linear feature interactions and its ensemble structure make it the more reliable and robust choice for a problem of this complexity. The decision tree (AUC 0.5336) and linear SVM (0.6204) trail behind, with the decision tree struggling without access to the high-information leakage columns. F1 scores (0.71–0.73) lagging behind accuracy (0.80–0.81) confirm imbalance, with models showing notably low precision (0.64) relative to recall (0.80), indicating frequent false positive delay predictions.

C. Regression Results (With Leakage Features)

Including all of the delay features shows reasonable performance for most of the models. An MAE of around 10 minutes shows that the predictions are reasonable on average, and the RMSE near 40 minutes implies that there are some extreme predictions not near the actual delay values. The R^2 is moderate at around 0.8, meaning that the model explains about 80% of the variation in delay times. The train (Table 5) and test (Table 6) performances are similar, so there is no overfitting observed.

The exception to these trends is linear regression. It has almost perfect predictions, with an RMSE of 0.63 minutes, MAE of 0.45 minutes, and R^2 of 1.0000 on the test set.

Table 5. Training set performance for regression, with leakage features.

Model	RMSE (minutes)	MAE (minutes)	R^2
Decision Tree	39.94	10.56	0.8302
Random Forest	40.39	12.49	0.8264
Gradient Boosted Trees	34.03	7.70	0.8769
Linear Regression	0.67	0.45	1.0000

Table 6. Test set performance for regression with all features (leakage included).

Model	RMSE (minutes)	MAE (minutes)	R ²
Decision Tree	42.40	10.88	0.8079
Random Forest	42.10	12.69	0.8107
Gradient Boosted Trees	41.38	8.59	0.8164
Linear Regression	0.63	0.45	1.0000

This set of features is more reasonable for predicting flight delays after the flight is in progress, although many of these features are currently only available after the flight has been completed. Dep_Delay and Delay_LastAircraft could be collected right before the flight begins, while delay-specific features could be updated as the flight progresses and be provided to the airlines during the flight process instead of after if possible.

D. Regression Results (Without Leakage Features)

Investigating the regression results further, especially for linear regression, we tried removing delay-specific features: Dep_Delay, Delay_Carrier, Delay_Weather, Delay_NAS, Delay_Security, and Delay_LastAircraft. This is an accurate setup to predict delays before the flight starts, but it led to a major reduction in performance for all of the models (Tables 7 and 8).

Table 7. Training set performance for regression, without leakage features.

Model	RMSE (minutes)	MAE (minutes)	R ²
Decision Tree	94.83	49.77	0.0434
Random Forest	94.37	49.72	0.0535
Gradient Boosted Trees	89.15	47.82	0.1552
Linear Regression	95.42	50.30	0.0322

Table 8. Test set performance for regression without leakage features (realistic pre-flight scenario).

Model	RMSE (minutes)	MAE (minutes)	R ²
Decision Tree	95.50	49.81	0.0238
Random Forest	94.72	49.67	0.0362
Gradient Boosted Trees	94.74	49.23	0.0356
Linear Regression	95.03	50.07	0.0297

Train and test results are still similar, but now the models typically have an RMSE of 95 minutes, MAE of 50 minutes, and R² of 0.03. This translates to delay predictions being about an hour off with some even more extreme cases. The R² confirms the models' bad performance with a value near 0, meaning that the models are not sufficiently capturing the variations in delay.

In this test, GBT is the optimal model because of its low MAE compared to the other models, giving it the best predictions overall. At the same time, it has a similar RMSE and R² as the other models.

With this set of features, linear regression has a similar performance to the other models. This means that it was heavily relying on a linear combination of the delay-specific features for its previous near-perfect performance. The

decrease in performance across all of the models when removing delay features shows that accurate delay predictions are highly dependent on day-of-delay information.

E. Regression Results (Iterative Improvement)

All four models collapse to R² values near 0.03, with MAEs around 49–50 minutes and RMSEs near 95 minutes. In other words, without same-day operational signals such as Dep_Delay and the per-cause delay attributions, exact delay duration is essentially unpredictable from the available pre-flight features. This is consistent with the structure of delay causation: most of the variance in delay magnitude is driven by events that unfold on the day of travel — gate availability, crew timing, upstream aircraft routing, and evolving weather — rather than by features known the day before.

To address this, we developed an iterative-improvement track combining cyclical encoding of temporal features, target encoding of high-cardinality categorical variables, and Bayesian hyperparameter optimization on a LightGBM regressor. After 30 Optuna trials, this track improved MAE from a baseline of 48.06 to 44.02 minutes and lifted R² from 0.105 to 0.262 (Table 9) — a meaningful gain, but still indicating that more than 70% of the variance in delay magnitude is unexplained by pre-flight features.

Table 9. Result of iterative improvement.

Stage	RMSE (minutes)	MAE (minutes)	R ²
Baseline	91.53	48.06	0.105
Feature Engineering	91.49	48.08	0.106
Fine Tuning	83.13	44.02	0.262

Motivated by the heavy tail of the delay distribution, we then implemented a two-stage hybrid architecture. A Tier-1 classifier first separates "severe" delays (>143 minutes, the top 10% of delayed flights) from the normal range, and a Tier-2 regressor predicts exact duration only on the normal-range subset, while flagging severe-delay candidates with an anomaly alert rather than a point estimate. On the normal-range subpopulation, the Tier-2 regressor achieves an MAE of 20.75 minutes — a substantial improvement over the ~49-minute MAE of the single-stage models — at the cost of declining to produce a duration estimate for the most extreme delays, which we argue are inherently unpredictable from pre-flight information alone.

IX. DISCUSSION

The most important finding of this study is the magnitude of the gap between with-leakage and without-leakage performance. With leakage features included, all four classifiers achieve AUC-ROC near 0.999, and linear regression effectively reconstructs the delay duration with sub-minute error. Once those columns are removed, AUC-ROC drops to roughly 0.66 for the best classifier, and regression R² collapses from 0.81 to roughly 0.03 for all four base models. This gap is entirely attributable to the post-arrival nature of the leakage columns and underscores how easily reported flight-delay model performance in the literature can be inflated when feature-time alignment is not enforced.

For the realistic pre-flight scenario, random forest emerges as the most reliable classifier. Its AUC-ROC of 0.665 is essentially tied with logistic regression, but its

ensemble structure and ability to capture non-linear feature interactions make it more robust to the noisy, mixed-type feature space typical of operational flight data. The class-imbalance signature in the metrics — accuracy clustered near 80%, F1 near 0.71, precision substantially lower than recall — reflects the underlying base rate of delays (about 20%) and indicates that models are predicting "delayed" frequently enough to recover most true positives but at the cost of substantial false positives.

For the regression task, the practical lesson is that exact delay duration is not learnable from pre-flight information alone. The two-stage architecture — outlier separation plus normal-range regression — is the most defensible deployment strategy, since it converts the long-tail problem into two more tractable subproblems and provides honest uncertainty signaling for severe delays rather than over-confident point estimates.

A. Model Limitations

The classification models have several limitations. The near-perfect all-features performance is entirely due to data leakage, as post-arrival columns like `Arr_Delay` are unavailable at departure time, making that configuration unsuitable for real deployment. In the no-leakage track, AUC-ROC values of 0.53-0.67 reflect weak pre-flight signal, and additional operational data such as gate status and air traffic congestion would likely be needed to improve classifier performance. Class imbalance is also a concern, as 80% accuracy with F1 near 0.71 suggests a majority-class baseline would perform similarly, and no resampling techniques were applied. Finally, model settings such as tree depth and regularization strength were chosen manually rather than through systematic tuning, as exhaustive hyperparameter search was not feasible on the free Databricks tier due to the computational cost of evaluating all parameter combinations across such a large dataset.

The regression models share these constraints and add several of their own. R^2 values near 0.03 in the no-leakage track indicate that pre-flight features simply do not contain enough information to recover the exact delay magnitude, particularly for the long tail of severe delays driven by same-day operational events. The two-stage architecture mitigates but does not eliminate this gap, and its 143-minute severe-delay threshold was chosen empirically rather than through formal cost-sensitive analysis. Both modeling tracks would benefit from richer pre-departure features — real-time air-traffic flow, upstream aircraft delay status, and short-horizon weather nowcasts.

X. CONCLUSION

This project developed and evaluated a two-stage machine learning system for predicting US domestic flight delays, comprising a binary classifier for delay occurrence and a regressor for delay magnitude. By deliberately separating with-leakage and without-leakage feature configurations, we produced an honest characterization of how much of the apparent predictive performance in flight delay datasets is attributable to post-arrival data leakage versus genuine pre-flight signal — and the answer is that the gap is large.

In the realistic pre-flight setting, random forest is the most reliable classifier, achieving an AUC-ROC of 0.665 and

demonstrating robustness to the non-linear, mixed-type feature space. For regression, single-stage models are essentially uninformative without same-day operational data; the most defensible deployment is a two-stage hybrid that separates severe-delay outliers from the normal range, achieving an MAE of approximately 20.75 minutes on normal-range delayed flights while issuing anomaly alerts for the long tail.

Beyond the modeling outcomes, the project also delivers actionable, data-grounded recommendations to passengers: prefer morning, mid-week, non-summer departures; favor consistently on-time carriers (Republic, Endeavor) over chronic underperformers (JetBlue, Frontier); and where routing allows, choose mid-sized Midwest or Mountain West airports over Florida hubs. Future work should focus on incorporating real-time operational signals — gate and ramp status, en-route air traffic flow, and short-horizon weather nowcasts — and on systematic hyperparameter optimization beyond the constraints of the free Databricks tier, both of which we expect would meaningfully improve the realistic, leakage-free performance of the system.

References

- [1] KXAN. Which Airports Had the Most Delays and Cancellations in 2023? Available online: <https://www.kxan.com/news/national-news/which-airports-had-the-most-delays-and-cancellations-in-2023/> (Accessed on May 7, 2026).
- [2] C. J. Goodman and J. D. Small Griswold, "Meteorological impacts on commercial aviation delays and cancellations in the continental United States," *J. Appl. Meteorol. Climatol.*, vol. 58, no. 3, pp. 479–494, 2019.
- [3] M. E. Biswas, T. Sultana, A. K. Mandal, M. G. Morshed, and M. D. Hossain, "Spatio-temporal feature engineering and selection-based flight arrival delay prediction using deep feedforward regression network," *Electronics*, vol. 13, no. 24, p. 4910, 2024.
- [4] R. Henriques and I. Feiteira, "Predictive modelling: Flight delays and associated factors, Hartsfield–Jackson Atlanta International Airport," *Procedia Comput. Sci.*, vol. 138, pp. 638–645, 2018.
- [5] L. Xu, X. Yuan, Y. Liang, S. Yin, and Y. Wu, "Aeolus: A Multi-structural Flight Delay Dataset," in *Advances in Neural Information Processing Systems*, vol. 38, 2026.
- [6] J. Ajayi, Y. Xu, L. Li, and K. Wang, "Enhancing flight delay predictions using network centrality measures," *Information*, vol. 15, no. 9, p. 559, 2024.
- [7] J. L. Franco, M. V. M. Neto, F. A. Verri, and D. R. Amancio, "Graph machine learning for flight delay prediction due to holding maneuver," *Physica A: Statistical Mechanics and its Applications*, Art. no. 131318, 2026.
- [8] T. Wang and S.-C. Chen, "Multi-task local-global graph network for flight delay prediction," in *2022 IEEE 23rd International Conference on Information Reuse and Integration for Data Science (IRI)*, 2022, pp. 49–54.
- [9] Bordanova, "2023 US Civil Flights: Delay, Meteo and Aircraft," Kaggle, 2024. Available online: <https://www.kaggle.com/datasets/bordanova/2023-us-civil-flights-delay-meteo-and-aircraft>. (Accessed on May 7, 2026)
- [10] Apache Software Foundation. PySpark and MLlib Documentation. Available at: <https://spark.apache.org/docs/latest/ml-guide.html>
- [11] Databricks, Inc. Databricks Community Edition Documentation. Available at: <https://docs.databricks.com/>

- [12] Federal Aviation Administration (FAA). On-Time Performance — Reporting Operations. U.S. Department of Transportation. The FAA defines a delayed flight as one that arrives or departs 15 or more minutes after its scheduled time.
- [13] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019.
- [14] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems, vol. 30, 2017.

Individual Contributions

- Nayan: Data Preparation, Feature Selection, Classification
- Young Soo: EDA, Data Cleaning, Trends and Flight Recommendations, Iterative Improvement
- Jenna: EDA, Feature Engineering and Selection, Regression